

深入理解 AI Agent：設計原理與工程實踐

後記 後記：回到 Agent = LLM + 上下文 + 工具
{.unnumbered}

李博杰 著 · 繁體中文版

大字寬鬆排版 正文 14pt／行高 1.9

本書開篇提出了一個公式：**Agent = LLM + 上下文 + 工具**。全書十章，都在這三個詞裡展開。

第一章建立公式的三層理解——實現層、直覺層、學術層，並給出從工作流到自主 Agent 的編排光譜。隨後的章節沿「建構—評估與進化—互動與協作」逐步展開。

- **建構 Agent（第二至五章）**。上下文工程決定 Agent 在一次任務中看到什麼，記憶與知識庫把資訊擴充套件到多次會話；工具定義它能做什麼，程式碼生成則提供創造新工具與新系統的元能力。
- **評估與進化（第六至八章）**。評估把表現變成可信訊號，後訓練把高維能力寫入模型參數，持續進化再把生產經驗轉化為知識、指令、程式或參數的受控更新。
- **互動與協作（第九至十章）**。多模態與即時互動把感知和行動擴充套件到語音、GUI 與物理世界，多 Agent 協作進一步改變上下文、工具與責任的組織方式。

這三個層次並不是相互獨立的書架。第八章尤其依賴前文的全部基礎：沒有軌跡和知識系統，經驗無處保存；沒有程式碼能力，Agent 無法修改工具與 Harness；沒有評估，系統更無法判斷一次修改究竟是進步還是退化。它由此成為全書從「怎樣建構 Agent」轉向「怎樣讓 Agent 長期變好」的匯合點。

兩朵烏雲 {.unnumbered}

1900 年，開爾文說物理學晴朗的天空上還飄著兩朵烏雲——後來，一朵變成了相對論，另一朵變成了量子力學。今天 Agent 的天空同樣稱不上晴朗，我也看到兩朵烏雲。

第一朵烏雲，是 Agent 如何流式地、即時地與環境互動。 今天絕大多數 Agent 仍是按輪次（turn-by-turn）的「請求—應答」模式：你說完一句，它想一整段，再拋棄式吐出結果。但真實世界不會停下來等它想完——話會被打斷，畫面在持續變化，郵件在不斷到達。一個真正「活著」的 Agent，應該能邊聽邊想、邊說邊想，能在你話說到一半時就開始規劃，也能在沒人吩咐時主動發現「這封郵件該處理了」。走向這種即時性有兩條路，往往並行推進：一是**架構上做快慢分離**——即時與智慧幾乎是兩條正交的軸，單一模型難以兼顧，於是讓前臺快模型維持對話節奏、後臺慢模型負責深度思考；二是**把推理本身做快**——當 decode 速度足夠高，按輪次的等待就短到近乎消失，turn-by-turn 與「即時」的界限也隨之模糊。這條路正被晶片與推理引擎快速推進：小米 MiMo 已讓一個 1T 引數模型在單個 8 卡節點上把生成速度推過 1000 token/s^[1]，而把整個模型直接固化進晶片的專用方案（如 Taalas HC1）更是把 80 億引數模型推到約 17000 token/s、響應低於 100 毫秒^[2]。當模型每秒能吐出上千個字，「想完再說」和「邊想邊說」的體驗差距就被抹平了。

第二朵烏雲，是 Agent 如何像人一樣，從與環境互動的成功與失敗中持續積累經驗。 今天的模型更像一個記性極好、卻學不會新東西的天才：訓練時把人類知識背得滾瓜爛熟，上崗後卻幾乎不再成長——每次任務結束，那些踩過的坑、試出來的竅門，大多隨著上下文一起被丟掉。這究竟是不是真問題，取決於兩種針鋒相對的假設。

一種是**「小世界假設」**：一個足夠大的模型——比如幾兆引數——本就裝得下物理世界裡幾乎所有重要的通用知識，學一次就夠。持這種看法的人（不乏 OpenAI、Anthropic 的研究者）會指出，AI 今天唯獨在程式設計上最強，並不是因為程式碼對模型有什麼特殊，而是因為程式設計是人類最開放的領域：海量開原始程式碼擺在那裡，可供學習；而絕大多數產業壓根沒有公開的資訊與資料。於是前沿實驗室真正在做的，是一家家去與各行各業合作，把

各自的專業能力「蒸餾」進同一個大模型。按這種觀點，瓶頸既不在模型的容量、也不在它學不學得會，而在資料夠不夠——把資料喂進去、訓練一次，問題就解決了。

但**「大世界假設」**指向了單靠「訓練一次」補不上的一層：屬於某個具體使用者、某家具體公司的知識。你所在公司的程式碼規範、你團隊做 PPT 的口味、某個客戶特有的脾氣——它們不在任何訓練語料裡，而且時時在變；要貼合這個由無數具體情境拼成的「大世界」，模型只能在上崗之後持續學習，沒法指望出廠時一次配齊。這正是第三章的記憶與第八章的持續進化在摸索的方向：把經驗寫成知識文件、指令或程式，還是經過篩選後用於更新模型參數？更進一步，「AI for AI」和「AI for Science」都在推動 Agent 走到沒有現成答案的前沿；在那裡，它只能從一次次實驗的成敗中自主學習，而不是事事回頭問人。所以，模型最強的能力，終將不是記住，而是學習與適應。

這兩朵烏雲，都不是靠某一次模型升級就能憑空吹散的。要理解它們最終會怎樣被跨越，得先看清一件事：模型和 Agent，從來不是上下游，而是一起往前走的。

模型與 Agent 的共同演進 {.unnumbered}

回頭看那些 harness 裡層層疊疊的兜底邏輯——多級上下文壓縮、失敗數千次才熔斷的重試、悲觀地預設「不安全」的權限判斷——每一段看似醜陋的「屎山」，記錄的都是模型此刻還做不穩的地方。當下一代模型把這些約束內化，對應的程式碼就可以刪掉；而模型之所以能內化，又正是因為 Agent 早已在真實業務裡替它把這些坑趟了一遍，沉澱成了下一輪訓練的訊號。使用者提出真實的難題，應用層用 harness 把模型暫時做不好的事補上，這些補救再反過來變成模型下一次迭代的訓練訊號——這是一條自我強化的飛輪，兩朵烏雲正是靠它一點點被推著散去。

這條飛輪，也回答了第一章懸下的那個問題：**模型會不會最終吃掉 Harness？會，一層地吃。** 模型每穩定內化一種能力，對應的 Harness 層就可以刪掉——第九章的互動模型就是這麼樣本：打斷、插話這些曾經要靠外掛 harness 才能拼出的行為，如今被直接做進了模型內部。但這個「吃」永遠不會完結。一是訓練以月計，模型等得起，業務等不起；二是模型無法內化真實業務中所有的約束與偏好，總有一層最新的邊界需要外部邏輯兜底；三是每一代模型都會開啟新的能力前沿，而前沿處恰恰是模型最做不穩的地方。所以 Harness 不會消失，它只是隨著模型，不斷向新的前沿遷移。這也正是《苦澀的教訓》在 Agent 時代的讀法：通用方法終將勝出，但「終將」二字裡的每一段路，都是 Harness 鋪出來的。

而飛輪轉得最快的地方，是同時握住兩端的人。Anthropic 用 Claude Code 做的，正是讓自家模型和自家 harness 互相餵養、共同進化：模型知道 harness 會怎樣呼叫它，harness 也清楚模型的邊界在哪，兩端的每一次改動都能立刻回饋給對方。曾有人做過一個實驗，不換模型、只改 harness，任務準確率就從 52.8% 跳到 66.5%——這既說明 harness 今天的槓桿有多大，也提醒你：它之所以有這麼大槓桿，恰是因為模型還沒走到那一步。也正因如此，這條飛輪本身，就是這個時代最深的一條護城河：真實業務、回饋資料與模型迭代咬合得越緊，別人越難從外部追上。

這對你意味著什麼，取決於你站在飛輪的哪一端。如果你在造模型，護城河就是把這條飛輪轉起來——讓真實場景的回饋儘快迴流到訓練裡。如果你在模型之上造應用，harness 是你短期最鋒利的技術槓桿，但要清醒：模型每內化一層約束，就會順手抹平一批只靠 harness 建立的優勢。應用層真正長久的護城河，往往在技術之外——獨佔的資料、穩固的渠道、使用者的信任、網路效應，以及那些 AI 短期內接不住、必須由人與 Agent 協作才能完成的複雜場景。把 harness 用來爭取時間，把這段時間用來構築技術之外的壁壘，才是穩妥的打法。

所以，不必焦慮手裡的框架會不會過時。模型每幾個月迭代一次，具體的 API、產品和榜單都會翻篇，但「看到什麼、能做什麼、如何驗證做得對不對」這三個問題不會過時——它們描述的不是某個模型的用法，而是智慧系統與世界互動的基本方式。掌握了它們，無論下一代模型帶來什麼新能力，你都知道該把它放進公式的哪個位置，也能一眼看出，它離吹散那兩朵烏雲還有多遠。

Agent 技術仍在飛速演進，一本書追不上所有變化。但如果這本書讓你帶走的不是某個 API 的具體用法，而是一套能在技術浪潮裡保持清醒的判斷力，那它就完成了使命。本書全部正文、配圖與配套實驗程式碼都是開源的，歡迎你去倉庫裡把實驗親手跑一遍、提 issue 和 PR——讀懂和做出來之間，隔著一條只能靠雙手跨過的河。而 Agent 最迷人的地方，正在於它能透過寫程式碼創造新的能力，甚至改進自己；讀到這裡，你已經握住了「創造」的原則。接下來，去造點什麼吧。

註釋

1. 小米 MiMo-V2.5-Pro-UltraSpeed 透過 FP4 量化、DFlash 並行推測解碼與 TileRT 推理系統的模型—系統協同設計，在單個通用 8-GPU 節點上首次把 1T 引數模型的生成速度推過 1000 token/s。見小米 MiMo 官方技術部落格「Pushing 1T-Parameter Model Generation Speed to 1000 TPS」，2026. <https://mimo.xiaomi.com/blog/mimo-tilert-1000tps>
2. Taalas HC1 將 Llama 3.1 8B 整個模型固化進 6nm 晶片，實現約 17000 token/s、響應低於 100 毫秒；代價是晶片只能執行被固化的那個模型，模型更新需重新流片。見 Karl Freund, “Taalas Launches Hardcore Chip With ‘Insane’ AI Inference Performance,” Forbes, 2026. <https://www.forbes.com/sites/karlfreund/2026/02/19/taalas-launches-hardcore-chip-with-insane-ai-inference-performance/>。